

pysystemtrade -> backtest_SmallCaps

Documento técnico de traducción arquitectónica.

Objetivo:

- explicar cómo está estructurado **pysystemtrade**
- identificar qué patrón resuelve cada capa
- mapear esas capas al proyecto **backtest_SmallCaps**
- dejar claro qué conviene copiar, qué conviene adaptar y qué no conviene heredar

No es un documento de marketing ni de “buenas prácticas” genéricas. Es una comparación de ingeniería entre dos sistemas con dominios distintos:

- **pysystemtrade**: futuros, posiciones, ejecución continua, contract rolls
- **backtest_SmallCaps**: micro/small caps, lifecycle de ticker, QA de dataset, calidad de evento, event-driven research

1. Qué es **pysystemtrade** en términos estructurales

pysystemtrade no es solo un backtester. Es un sistema completo con cuatro capas técnicas bien separadas:

data/storage

->

simulation/backtest system

->

production intent generation

->

execution and reconciliation

Más en detalle:

sysdata

abstrae almacenamiento y acceso a datos

systems

calcula reglas, forecasts, tamaños, cartera, cuentas

sysproduction

orquestra procesos diarios y mantenimiento operativo

sysexecution

convierte intención de trading en órdenes reales y reconcilia fills

La idea central no es “tener muchas clases”. La idea central es:

- cada capa tiene un contrato claro
- cada capa ve la capa inferior a través de una interfaz estable
- la producción no llama directamente a piezas arbitrarias del backtest
- existe un paso intermedio entre “salida del modelo” y “orden al broker”

2. Núcleo conceptual de pysystemtrade

El núcleo es el objeto **System**.

Un **System** es una pipeline de **stages**. Cada **stage**:

- tiene un nombre fijo
- expone métodos esperados
- consume datos producidos por otras **stages**
- puede cachear resultados

Esquema conceptual:

```
rawdata
->
rules
->
forecastScaleCap
->
combForecast
->
positionSize
->
portfolio
->
accounts
```

Eso significa:

- **rawdata** prepara series base
- **rules** calcula señales individuales
- **forecastScaleCap** normaliza y limita señales
- **combForecast** combina señales
- **positionSize** transforma forecast en tamaño
- **portfolio** aplica lógica de cartera
- **accounts** calcula PnL, métricas, etc.

Punto importante:

- no es un DAG genérico libre
- es una cadena deliberadamente opinionated
- esa rigidez ayuda a mantener consistencia entre backtest y producción

3. Capa de datos en pysystemtrade

La capa **sysdata** resuelve dos problemas distintos:

1. dónde vive el dato físicamente
2. qué interfaz usa el sistema para leerlo

Eso se resuelve con:

- clases de almacenamiento concretas
- **dataBlob**

- `simData`

Esquema:

```
storage class
  csvSomethingData
  parquetSomethingData
  mongoSomethingData
  ->
dataBlob
  ->
simData / dbFuturesSimData / csvFuturesSimData
  ->
System
```

Interpretación:

- las clases concretas saben leer/escribir en CSV, Parquet, MongoDB, etc.
- `dataBlob` agrega esas clases y las expone con una interfaz lógica uniforme
- `simData` es lo que el `System` consume

Decisión de diseño muy importante:

- MongoDB se usa para datos estáticos / metadata / estado
- Parquet se usa para time series

Ventaja real de esta arquitectura:

- el motor de sistema no depende de “si el dato viene de CSV o Parquet”
- el almacenamiento puede cambiar sin reescribir el cálculo de señales

4. Backtest en `pysystemtrade`

El backtest no está diseñado como “script que corre todo de arriba abajo”. Está diseñado como:

```
System(config, data, stages)
  ->
pedidos de cálculo por método
  ->
cache interno
  ->
resultados reutilizables
```

Ejemplo conceptual:

```
system.accounts.portfolio().sharpe()
```

Eso dispara internamente:

- `portfolio`
- que depende de `positionSize`
- que depende de `combForecast`
- que depende de otras etapas

La caché permite:

- recalcular solo lo necesario
- guardar resultados intermedios
- persistir parte del estado a pickle

Esto es útil para:

- investigación iterativa
- diagnósticos
- reruns costosos

Límite importante:

- la caché es útil en un mundo de sistemas relativamente estables por instrumento
- no es una solución universal para pipelines de QA de datos masivos y artefactos event-driven

5. Producción en `pysystemtrade`

La producción está separada del backtest. No se envían órdenes “directamente desde el objeto `System`”.

Cadena:

```
run_systems
->
run_strategy_order_generator
->
run_stack_handler
->
broker
```

5.1 `run_systems`

Responsabilidad:

- ejecutar el cálculo operativo de la estrategia
- producir la intención de trading para el día

En la estrategia clásica:

- recalcula buffers / target positions por instrumento
- deja el resultado listo para la siguiente capa

No hace:

- ejecución a broker
- reconciliación de fills intradía

5.2 `run_strategy_order_generator`

Responsabilidad:

- convertir intención de trading en `instrument orders`

Esto es crucial. Aquí aparece una separación que merece copiar:

```
model output
!=
executable order
```

El generador:

- mira posición objetivo
- mira posición actual
- decide qué cambio hay que hacer
- coloca órdenes internas a nivel instrumento

Todavía no son órdenes de broker.

5.3 run_stack_handler

Responsabilidad:

- proceso vivo intradía
- manejar tres niveles de órdenes
- reconciliar fills

Jerarquía:

```
instrument order
->
contract order
->
broker order
```

Y al volver:

```
broker fill
->
contract fill
->
instrument fill / position update
```

Esto resuelve un problema real de producción:

- la lógica de inversión piensa en instrumentos/estrategia
- la ejecución real piensa en contratos concretos y restricciones de broker

En futuros eso es indispensable por:

- expiries
- rolls
- calendar spreads
- diferencias entre contrato “priced” y contrato “forward”

6. Herramientas operativas de pystemtrade

El sistema no presupone que todo va perfecto. Incluye herramientas de control manual:

- interactive_controls
- interactive_diagnostics

- `interactive_order_stack`
- herramientas de rolling
- reporting
- dashboard
- backup y housekeeping

Traducción conceptual:

```
core model
+
ops controls
+
inspection tools
+
state repair / override tools
```

Eso es importante porque en sistemas reales:

- siempre hay estados intermedios
- siempre hay reconciliaciones
- siempre hay datos/órdenes que requieren intervención

7. Qué problemas resuelve bien `pysystemtrade`

Lo que resuelve muy bien:

1. separar cálculo de señal de ejecución real
2. separar storage físico de interfaz lógica de datos
3. formalizar un pipeline estable de cálculo por etapas
4. introducir jerarquía de órdenes entre intención y broker
5. dar herramientas de control operativo y reconciliación
6. mantener un modelo mental coherente entre research y producción

Esto no es superficial. Es el verdadero valor arquitectónico del proyecto.

8. Qué problemas **NO** resuelve `pysystemtrade` para vuestro caso

Aquí está la parte crítica.

`pysystemtrade` no está diseñado alrededor de:

- lifecycle de ticker
- listed/delisted/renamed/suspended
- universos point-in-time por equity microcap
- reconciliación quotes/trades/OHLCV
- QA de coverage por día/ticker
- diagnóstico de truncados, vacíos y artefactos vendor/pipeline
- event quality gates para pump/dump o microcap regimes

Su problema central es:

cómo calcular y ejecutar posiciones de futuros de forma robusta

El vuestro es:

cómo construir un dataset científicamente válido y útil para eventos micro/small-cap antes de cualquier decisión cuantitativa o de ejecución

Eso cambia casi todo:

- unidad de análisis
- tipo de dato
- fuente del riesgo metodológico
- punto donde aparece el error más grave

9. Diferencia de unidad de decisión

En `pysystemtrade`, la unidad de decisión práctica es algo así:

- estrategia x instrumento
- target position
- instrument order

En `backtest_SmallCaps`, la unidad de decisión correcta es algo así:

- ticker x día x ventana de vida
- calidad de cobertura
- comparabilidad entre fuentes
- usabilidad de evento
- clasificación de régimen / prefilter / gate

Eso significa que si copiáis la arquitectura, no debéis copiar la semántica interna.

10. Cómo traducir `pysystemtrade` a vuestro proyecto

La forma correcta de usarlo como referencia es esta:

```
pysystemtrade concept
->
equivalente deseable en backtest_SmallCaps
```

10.1 System por stages

En vuestro caso no debería convertirse en:

- rawdata -> rules -> portfolio

Debería convertirse en algo más parecido a:

```
universe_pti
->
official_lifecycle
->
dataset_materialization
->
data_integrity_gates
->
policy_integration
->
```

```
event_index
->
event_quality_gate
->
backtest_dataset
```

Lectura:

- el “motor” de vuestro sistema no es una cadena de forecasts
- es una cadena de certificación + selección + construcción de evidencia de evento

Esto sí conviene formalizar como stages.

10.2 dataBlob

Equivalente deseable:

```
storage adapters
  R2
  local parquet
  official reference files
  run artifacts
  ->
project data interface
  ->
stages / notebooks / agents
```

Hoy en vuestro proyecto muchas piezas acceden a paths directamente. Eso os da velocidad, pero os resta uniformidad.

Lo que convendría heredar de `pysystemtrade`:

- una interfaz de datos lógica por encima de:
 - parquet local
 - artefactos `runs/`
 - referencias oficiales
 - datos operativos de agentes

No hace falta copiar `dataBlob` tal cual. Sí conviene copiar la idea.

10.3 run_systems

Equivalente deseable en vuestro proyecto:

```
nightly / batch research build
->
recalcular universo usable
->
recalcular policy + event index + quality gate
->
materializar dataset congelado para backtest o siguiente proceso
```

Es decir:

- una capa que no descarga datos crudos
- no ejecuta broker
- no valida archivos uno a uno
- solo toma artefactos ya certificados y produce la “intención científica” del día/corrida

10.4 run_strategy_order_generator

Equivalente deseable:

```
research output
->
candidate events / candidate trades
->
execution intents
```

En vuestro dominio, eso podría ser:

- evento usable
- score
- régimen
- ventana
- entrada potencial
- constraints

Todavía no broker order.

Es el puente entre:

- dataset utilizable
- decisión operativa concreta

10.5 run_stack_handler

En vuestro caso aún no existe un equivalente maduro. Si más adelante vais a ejecución real, sí convendría introducir una jerarquía explícita:

```
signal intent
->
strategy order
->
venue/broker order
->
fill
->
position / state reconciliation
```

Aunque no tengáis contratos de futuros, la separación sigue siendo útil.

Porque en equities microcap también existe distancia entre:

- “quiero estar largo 20k USD”
- “qué órdenes se envían”
- “qué se llenó realmente”

- “qué posición tenemos de verdad”

11. Qué sí deberíais copiar

Copiar, adaptado:

1. contratos estrictos entre capas
2. separación entre research, intent generation y execution
3. interfaces de datos por encima del storage físico
4. artefactos persistidos por capa, no solo outputs finales
5. procesos operativos desacoplados
6. herramientas de inspección y control manual
7. reconciliación explícita entre estado esperado y estado observado

12. Qué no deberíais copiar

No copiar:

1. la semántica de futures / rolls / priced-forward contract
2. la pipeline forecast-centric
3. la idea de que el problema central es portfolio sizing
4. la dependencia conceptual de “instrument orders -> contract orders” tal cual
5. la suposición de que el dataset ya es suficientemente fiable antes del motor de decisión

Ese último punto es especialmente importante.

En pssystemtrade, la capa de datos es importante, pero no es el núcleo epistemológico del sistema.

En vuestro proyecto, sí lo es.

13. Diferencia filosófica central

pssystemtrade:

trusted market data

->

system model

->

portfolio intent

->

execution

backtest_SmallCaps:

raw vendor/local datasets

->

certification and reconciliation

->

event-usable dataset

->

research intent

->

optional execution layer later

Eso implica:

- en vuestro sistema, la capa “antes del modelo” es mucho más pesada y más crítica
- en `pysystemtrade`, la capa “después del modelo” es mucho más desarrollada

14. Arquitectura objetivo sugerida para vosotros

Si se traduce lo mejor de `pysystemtrade` a vuestro proyecto, la arquitectura recomendable sería:

Layer 0: Storage and references

- local parquet datasets
- official lifecycle
- market calendar
- run artifacts
- agent states

Layer 1: Data interfaces

- universe interface
- quotes interface
- ohlcv interface
- trades interface
- lifecycle interface
- run artifact interface

Layer 2: Certification stages

- PTI universe build
- lifecycle reconcile
- snapshot inventory
- schema validation
- time coverage
- ohlcv vs quotes
- condition / sequence / drift / spread / off-exchange

Layer 3: Policy stages

- prefilter
- manual review
- repair queue
- scale mismatch remediation
- final policy materialization

Layer 4: Event stages

- event index
- lifecycle scores
- regime classifier
- split/extreme audit
- event quality gate

Layer 5: Strategy / backtest layer

- candidate universe

- entry/exit logic
- fill model
- risk model
- portfolio aggregation if needed

Layer 6: Optional execution layer

- signal intent
- execution order
- broker order
- fills
- reconciliation

Eso sería la traducción correcta del espíritu de `pysystemtrade` a vuestro dominio.

15. Lectura final

Si alguien del equipo pregunta:

- “¿debemos copiar `pysystemtrade`?”

La respuesta técnica correcta es:

No como framework de trading.

Sí como referencia de arquitectura modular, contratos entre capas, separación research/production y reconciliación operativa.

Si alguien pregunta:

- “¿qué le falta a nuestro proyecto frente a `pysystemtrade`?”

La respuesta sería:

No le falta parecerse más a futures trading.

Le falta formalizar mejor las interfaces entre vuestras capas ya existentes y, cuando llegue el momento, separar claramente:

dataset certified

->

research intent

->

execution intent

->

execution state

Si alguien pregunta:

- “¿qué tenemos nosotros que `pysystemtrade` no tiene?”

La respuesta sería:

Una capa de certificación epistemológica de dataset mucho más profunda, necesaria por el dominio micro/small-cap event-driven.

16. Decisión práctica

Decisión recomendada:

- usar `pysystemtrade` como referencia de arquitectura de sistema
- no usarlo como molde directo de dominio
- reforzar en `backtest_SmallCaps`:
 - contratos de interfaces
 - separación de capas
 - procesos desacoplados
 - reconciliación explícita de estado
- mantener como rasgo diferencial propio:
 - PTI universe
 - lifecycle oficial
 - QA por evidencia
 - gates de evento
 - trazabilidad por artefacto y corrida